CI/CD Pipelines

build once, test, scan, promote, roll back

Watch the 4-min video

In one line: a pipeline verifies every change automatically (CI), builds one immutable artifact, and promotes that same artifact through staging to production with gates, health checks and a one-click rollback (CD), so releases become small, boring and repeatable.

1 Slow, scary and different every time

A developer builds on their laptop, copies it to a server and restarts it. It works, until the day a step is forgotten or the wrong version goes out, and nobody knows what is actually running. A CI/CD pipeline replaces those manual steps with an automated, repeatable path to production.

2 Continuous integration

Every change is merged often and verified automatically. On each pull request the pipeline builds and runs fast checks: unit tests, linting, type checks. Problems surface in minutes, while the change is fresh. A broken main branch is the team's top priority, because everyone builds on it.

3 Stages and gates

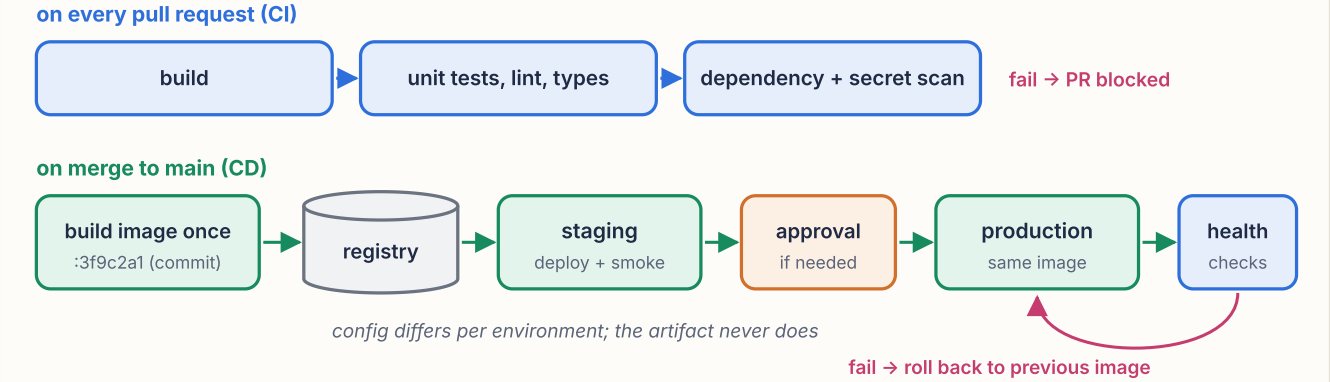


Figure 1. Each stage is a gate: if it fails, nothing moves forward. One image, tagged with the commit, is promoted from staging to production.

Build once, run unit and integration tests, **scan** dependencies for known vulnerabilities and the diff for leaked secrets, then package as an artifact (e.g. a Docker image) and push it to a registry. Later stages deploy, run smoke tests and check health.

4 One immutable artifact

Build the artifact **exactly once**, tag it with the commit (`orders:3f9c2a1`, never only `:latest`), and promote that same artifact from dev to staging to production. Only configuration changes between environments (lesson 38). Rebuilding per environment risks shipping something different from what you tested: a new base image, a moved dependency. The registry keeps every version, so you always know which commit is running and can redeploy the previous one.

5 Delivery vs deployment

	Continuous integration	Continuous delivery	Continuous deployment
Means	merge often, verify every change	every green build *could* be released	every green build *is* released
Prod release	n/a	a human presses the button	automatic, gated by tests and health
Needs	fast tests, trunk discipline	reliable pipeline, rollback	strong tests, monitoring, auto-rollback, flags (lesson 76)

Either way, production deploys need approvals where policy requires, post-deploy health checks, and a fast, **tested** rollback to the previous artifact. Rollout strategies (rolling, blue-green, canary) are lessons 73–75. Small, frequent releases are easier to verify and to undo.

6 The pipeline is production too

- **Pipeline as code** in the repo, reviewed like any change; branch protection and required reviews on main.
- **Short-lived credentials** via OIDC federation instead of long-lived cloud keys stored as secrets.
- **Pin third-party actions** to a commit SHA; a moved tag can run someone else's code with your credentials (lesson 58).
- **Fast:** cache dependencies, run independent jobs in parallel; an hour-long pipeline gets skipped.

```
on: {push: {branches: [main]}}
permissions: {id-token: write, contents: read}      # OIDC, no stored cloud secret
jobs:
  build:
    steps:
      - uses: actions/checkout@<pinned-sha>
      - uses: azure/login@<pinned-sha>                # federated credential
      - run: docker build -t $ACR/orders:${{ github.sha }} . && docker push $ACR/orders:${{ github.sha }}
  deploy-prod:
    needs: [build, deploy-staging]
    environment: production                          # required reviewers = approval gate
    steps:
      - run: kubectl set image deploy/orders orders=$ACR/orders:${{ github.sha }}
      - run: kubectl rollout status deploy/orders --timeout=5m || kubectl rollout undo deploy/orders
```

Figure 2. A sketch of the video's GitHub Actions → ACR → AKS flow (staging job omitted). The environment's required reviewers are the approval gate.

WORKED EXAMPLE: GITHUB ACTIONS TO AZURE KUBERNETES SERVICE

On each **pull request**: tests and a security scan. On **merge to main**: build a Docker image tagged with the commit SHA, push it to Azure Container Registry, deploy to **staging** automatically. After smoke tests pass and a reviewer approves the `production` environment, the **same image** is promoted to production. If `kubectl rollout status` times out or health checks fail, the pipeline runs `kubectl rollout undo` to return to the previous image.

COMMON MISTAKES

- Rebuilding the image for production, so prod runs something never tested.
- Deploying `:latest`: you can't tell what is running or roll back precisely.
- A rollback path that has never been exercised, discovered broken during an incident.
- Long-lived cloud admin keys in CI secrets and unpinned third-party actions.

7 Key terms

CI: merge often; build and test every change automatically

Continuous delivery: every green build is releasable

Continuous deployment: every green build is released automatically

Artifact: built, versioned output, e.g. an image

Promotion: moving the same artifact to the next environment

Gate: a check that must pass before the next stage

8 Interview questions

Q1. What should a production CI/CD pipeline include? BASIC

Build, test and security-scan stages, one immutable artifact promoted from dev to staging to production, approvals where needed, post-deploy health checks and a fast, tested rollback, with the pipeline itself secured and fast.

Q2. Why build the artifact only once? BASIC

So production runs exactly what you tested; rebuilding can pull different dependencies or base images.

Q3. Staging passed but production broke. What would you check in the pipeline? SCENARIO

That the same artifact was promoted, config differences between environments, missing prod-like data or traffic in staging, and whether health checks gate the rollout.

Q4. How do you let CI deploy to Azure without storing a secret? SCENARIO

Use OIDC workload identity federation: the workflow gets a short-lived token from GitHub's OIDC provider and exchanges it for an Azure token scoped to that repo and environment.

Q5. Continuous delivery vs continuous deployment? TRICKY

Delivery keeps every build releasable and a human decides when; deployment releases every passing build automatically.

Q6. Is a fast rollback a substitute for testing? TRICKY

No: it limits damage but users still see the failure, and data changes (migrations, sent emails) may not roll back. You need both.

Next: SD-N-015 · Rolling Deployments backendroadmap.com